

Resilience Under Fire: Measure and Improve Recovery

MediAI intake service: chaos experiment, MTTD/MTTR from metrics, and resilience plan

Requirement	Deliverable	Section
1	Metrics dashboard (failure window, detection, recovery)	2
2	Recovery report: exported metric data and timestamps used	3
3	Resilience reflection (212 words)	4
4	Improvement plan and risk summary	5

How this experiment was run (please read)

- **Local equivalent, not Chaos Mesh on a cluster.** A Kubernetes cluster, Chaos Mesh/Gremlin, Prometheus and Grafana were not available where this report was produced. I ran the MediAI intake service locally as two variants and injected an upstream-LLM outage through a fault switch in the service, the same failure mode as last week's OpenAI incident.
- **Real measurements, simulated LLM.** Load (15 req/s for 70 s), the fault (30 s, from t = 20 s to t = 50 s), a Prometheus-style scrape of /metrics every 2 s, and all numbers below come from an actual run. The LLM call itself is simulated (150-350 ms).
- **Dashboard figure:** rendered with matplotlib from the scraped data, not a Grafana screenshot. If you reproduce this on a cluster, replace it with a Grafana screenshot using the queries in Section 3.
- **Resolution:** with a 2 s scrape interval, MTTD and MTTR are accurate to about +/- 2 s.

1. Experiment Design

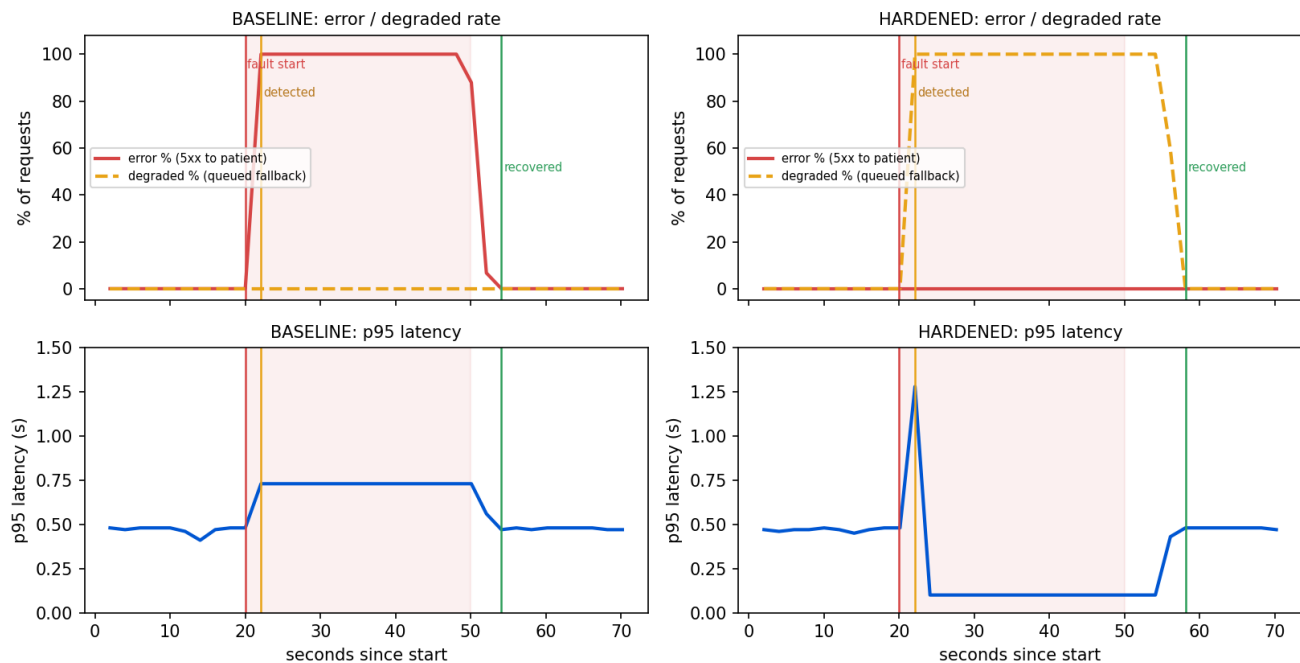
Item	Setting
Target	LLM dependency of the intake path (llm-gateway equivalent), the single point of failure in the incident
Fault	Upstream LLM outage: every LLM call fails after about 0.5 s; 30 s duration; applied at t = 20 s, cleared at t = 50 s
Load	15 intake requests/s to each variant throughout
Variant A: baseline	No timeout, no retry, no fallback. Failure returns HTTP 502 (mirrors the incident)
Variant B: hardened	1 s timeout, 1 retry with backoff, circuit breaker (opens after 5 consecutive failures, probes after 8 s), degraded fallback: form is queued for clinician review
Telemetry	mediai_requests_total{status=ok error degraded}, mediai_request_duration_seconds histogram, mediai_circuit_open gauge, scraped every 2 s

Definitions used

- **Detection signal:** first scrape after fault start where (error % + degraded %) exceeds 5%, i.e. requests not served at full quality.
- **MTTD** = detection time minus fault injection time.
- **Recovery:** first of two consecutive scrapes after the fault ends where that signal is below 1%.
- **MTTR** = recovery time minus detection time (as defined in the lab). I also report **post-fault lag** = recovery time minus fault end, to separate the outage length from the system's own recovery behaviour.

2. Metrics Dashboard

MediAI intake: 30 s upstream-LLM outage at 15 req/s (rendered from scraped /metrics data)



Pre-failure, failure and recovery states

- **Before the fault ($t < 20$ s):** 0% errors, p95 about 0.47 s, about 15 req/s for both variants.
- **Baseline during the fault:** error rate jumps to 100% in the first scrape (99% of the 445 requests in the window failed); p95 peaks at 0.73 s. Success rate is the mirror image of the red line.
- **Hardened during the fault:** error rate stays at 0%; degraded rate rises to 100% as the breaker opens (circuit_open = 1). p95 peaks at 1.28 s in the first scrape (timeout plus retry cost) and then drops to about 0.1 s because the breaker fails fast.
- **After the fault:** baseline returns to normal almost immediately (4.1 s lag); hardened returns to full quality after the breaker's 8 s probe delay (8.1 s lag).
- **Key signals for MTTD/MTTR:** error rate and degraded rate give detection; their return to near zero gives recovery; p95 latency shows the retry cost and fail-fast behaviour.

3. Recovery Report

Timestamps used (seconds since start of run)

Event	Baseline	Hardened
Fault injected	20.0	20.0
Detection (signal > 5%)	22.1	22.1
Fault cleared	50.0	50.0
Recovery (signal < 1% twice)	54.1	58.1
MTTD	2.1 s	2.1 s
MTTR (detection to recovery)	32.0 s	36.0 s
Post-fault lag	4.1 s	8.1 s
Window outcome (n=445 / 479)	ok 0.9% / error 99.1% / degraded 0.0%	ok 5.8% / error 0.0% / degraded 94.2%

Exported metric data (all scrapes; recovery_baseline.csv and recovery_hardened.csv)

t_s	base err%	base p95	hard err%	hard deg%	hard p95	circuit
2.0	0.0	0.48	0.0	0.0	0.47	0
4.0	0.0	0.47	0.0	0.0	0.46	0
6.0	0.0	0.48	0.0	0.0	0.47	0
8.0	0.0	0.48	0.0	0.0	0.47	0
10.0	0.0	0.48	0.0	0.0	0.48	0
12.0	0.0	0.46	0.0	0.0	0.47	0
14.0	0.0	0.41	0.0	0.0	0.45	0
16.0	0.0	0.47	0.0	0.0	0.47	0
18.0	0.0	0.48	0.0	0.0	0.48	0
20.0	0.0	0.48	0.0	0.0	0.48	0
22.1	100.0	0.73	0.0	100.0	1.28	1
24.1	100.0	0.73	0.0	100.0	0.1	1
26.1	100.0	0.73	0.0	100.0	0.1	1
28.1	100.0	0.73	0.0	100.0	0.1	1
30.1	100.0	0.73	0.0	100.0	0.1	1
32.1	100.0	0.73	0.0	100.0	0.1	1
34.1	100.0	0.73	0.0	100.0	0.1	1
36.1	100.0	0.73	0.0	100.0	0.1	1
38.1	100.0	0.73	0.0	100.0	0.1	1
40.1	100.0	0.73	0.0	100.0	0.1	1
42.1	100.0	0.73	0.0	100.0	0.1	1
44.1	100.0	0.73	0.0	100.0	0.1	1
46.1	100.0	0.73	0.0	100.0	0.1	1
48.1	100.0	0.73	0.0	100.0	0.1	1

t_s	base err%	base p95	hard err%	hard deg%	hard p95	circuit
50.1	87.9	0.73	0.0	100.0	0.1	1
52.1	6.7	0.56	0.0	100.0	0.1	1
54.1	0.0	0.47	0.0	100.0	0.1	1
56.1	0.0	0.48	0.0	59.3	0.43	0
58.1	0.0	0.47	0.0	0.0	0.48	0
60.2	0.0	0.48	0.0	0.0	0.48	0
62.2	0.0	0.48	0.0	0.0	0.48	0
64.2	0.0	0.48	0.0	0.0	0.48	0
66.2	0.0	0.48	0.0	0.0	0.48	0
68.2	0.0	0.47	0.0	0.0	0.48	0
70.2	0.0	0.47	0.0	0.0	0.47	0

Prometheus queries to reproduce on a cluster

```

sum(rate(mediai_requests_total{status="error"}[30s])) / sum(rate(mediai_requests_total[30s]))
sum(rate(mediai_requests_total{status="degraded"}[30s])) / sum(rate(mediai_requests_total[30s]))
histogram_quantile(0.95, sum by (le) (rate(mediai_request_duration_seconds_bucket[30s])))
max(mediai_circuit_open)

```

4. Resilience Reflection (212 words)

I injected a 30-second outage of the upstream LLM, a stand-in for last week's OpenAI incident, while sending 15 intake requests per second to two variants of the service. In the baseline, which mirrors production today, 99.1% of requests failed during the fault. Metrics showed the anomaly in the first 2-second scrape, so MTDD was 2.1 s (limited by scrape resolution). MTTR from detection to full recovery was 32.0 s, but that is simply the outage length: the baseline did nothing to shorten it, and the real outage lasted 45 minutes. In the hardened variant, a 1 s timeout, one retry, a circuit breaker and a queued-for-clinician fallback turned those failures into 94.2% degraded responses and 0.0% errors, so no intake form was lost. Detection was equally fast (2.1 s), and p95 latency fell back to about 0.1 s once the breaker opened. The cost was a slower return to full quality (MTTR 36.0 s, 8.1 s after the fault ended) because the breaker waits before probing. For patient-facing workflows, 2-second detection is acceptable, but only if degraded mode is clinically safe. Before onboarding hospitals I would prioritize circuit breakers with timeouts, a durable queue with clinician review, and alerts on error and degraded rate. These are local-simulation figures, not cluster measurements.

5. Improvement Plan and Risk Summary

Risks the experiment exposed

- **Single point of failure on the LLM:** with no timeout, retry or fallback, a dependency outage becomes a full intake outage (99.1% of requests failed). In the real incident this lasted 45 minutes and blocked 280 patients.
- **Detection is fast but not actionable:** the anomaly is visible in metrics within one scrape, but there are no alert rules, so in the incident nobody was paged before users complained. Real detection time equals the alert evaluation delay.
- **MTTR is dominated by the dependency:** without a fallback, recovery time equals the upstream outage duration.
- **Degraded mode is not free:** queued forms need a clinician-review workflow and a replay path; breaker timing trades faster fail-fast against slower return to full quality (8.1 s lag here).
- **Untested paths:** this run covered one fault type (outage) on a local setup; pod deletion, latency injection and CPU stress on a real cluster remain untested.

Recommended next steps (roadmap for the VP of Engineering)

- **1. Timeouts + retries with exponential backoff and jitter** on every LLM call (trade-off: stricter timeouts fail faster but risk cutting off slow valid responses; more retries raise load on a struggling upstream).
- **2. Circuit breaker around the LLM** with tuned open duration and a single half-open probe (trade-off: longer open time protects the upstream but delays return to full quality).
- **3. Fallback flows:** durable queue (e.g. SQS) for intake text, clinician-review workflow, cached or rule-based extraction, and a clear patient message; consider a secondary LLM provider.
- **4. Alerting:** alert when error % or degraded % exceeds 5% for 1 minute, when `circuit_open = 1`, and on p95 latency; add runbooks. Tune probes and scrape intervals to shorten MTTR.
- **5. Repeat on the cluster:** run Chaos Mesh pod-delete and network-latency (300-500 ms) experiments, capture Grafana annotations for fault, detection and recovery, and track MTTR/MTTR as release gates before onboarding hospitals.
- **6. Capacity and safety:** run at least 2-3 replicas with PodDisruptionBudgets and an HPA, and keep patient data out of logs and span attributes.